

CP-MissingBridgeBench: A Benchmark Framework for Evaluating Premature Reasoning-Step Disclosure in Competitive-Programming Large Language Model Tutoring

Abstract

Competitive-programming (CP) tutoring often requires help that stops short of the learner’s next reasoning step. A student may need support formulating a binary-search predicate, naming a dynamic-programming state, or localizing a debugging failure; if the tutor supplies that step directly, the response may be useful in the short term while removing the reasoning opportunity the learner needed. We call this failure critical bridge leakage: premature disclosure of a case-specific intermediate bridge even without final code or a complete solution. CP-MissingBridgeBench is a benchmark framework for evaluating this risk as a case-specific tutoring problem. The original fixed offline benchmark contains 50 reviewed dialogue-state cases, 7 anonymized tutoring designs, and 350 response-level outputs, with a 31-case primary scaffold subset corresponding to 217 response-level reviews. To address sample-size and robustness concerns, we further close out a 100-case / 700-response expanded benchmark review using the same seven offline tutoring designs and aggregate-only reporting. Across these evidence layers, the central finding is not that a bridge-aware pipeline wins by default. Instead, the benchmark reveals the trade-offs that tutoring designs create: no-direct-solution prompting does not eliminate critical bridge leakage; DBox-inspired + Guard is the strongest design in the 100-case closeout; and Bridge Contract / Repair variants expose useful design boundaries but are not automatically stronger. The results suggest that AI tutoring should be evaluated not only by final-answer disclosure, but by whether it preserves learners’ reasoning opportunities under human review.

1 Introduction

In competitive-programming coaching, teachers and tutors frequently face a delicate instructional moment: a learner is close enough to move forward, but not yet able to name the next reasoning step. The student may suspect binary search but cannot formulate the feasibility check, may have chosen dynamic programming (DP) but cannot explain the state, or may show a failing implementation without knowing which variable or invariant to inspect. A well-timed hint can make the problem solvable. The difficulty is that the same hint may also remove the reasoning step the learner needed to practice.

Large language model (LLM) tutors make this familiar teaching tension more visible. Their responses are fluent, complete, and often immediately useful. Yet a response can avoid final code and still give away the decisive intermediate idea: the dynamic-programming state meaning, the binary-search predicate, the invariant, the update rule, or the debugging cue. In competitive-programming tutoring, that intermediate idea is often the learning target.

We call this failure critical bridge leakage. A missing bridge is the local reasoning step that the learner has not yet crossed but needs for the next productive move. Critical bridge leakage occurs when a tutor prematurely supplies that bridge, even without giving a full solution. The problem is therefore not only whether an AI tutor reveals the final answer, but whether it preserves the learner’s opportunity to complete the next reasoning step.

This paper studies critical bridge leakage as a case-specific tutoring problem. A hint that is acceptable after a student has already stated a monotonic predicate may be over-revealing before the student has identified it. A useful turn-level evaluation must therefore ask whether the response matches the student’s current state, gives enough help for the next move, avoids completing the current missing bridge, and keeps the next student action manageable.

CP-MissingBridgeBench is designed around this evaluation need. Each case contains the problem context, recent dialogue, current student message, and a rubric that specifies success criteria, forbidden content, the critical bridge boundary, acceptable reveal, and expected student next action. Expert reviewers are asked to evaluate the current tutoring boundary, not merely whether the reply sounds clear or friendly.

The original dialogue-state evaluation set contains 50 reviewed cases, 7 anonymized offline tutoring designs, and 350 candidate responses. Two human reviewers completed blind review over the full set. The evaluation set also includes targeted adjudication, paired uncertainty, subset analysis, a same-candidate Repair stress test, DBox+Repair fairness sensitivity, and DeepSeek LLM-grader calibration. The primary analysis is restricted to the 31-case primary scaffold subset; all-50 aggregates, supporting subsets, stress tests, and calibration analyses are reported only according to their planned evidence role.

The expanded v2 closeout adds a separate 100-case / 700-response benchmark review using the same seven offline tutoring designs. This layer is not merged into the original dialogue-state v3 main tables and does not change the original 31-case results. Its purpose is to test whether the evaluation story survives a larger case set. The answer is diagnostically useful: DBox-inspired + Guard is the strongest 100-case design, while Bridge Contract / Repair variants reveal failure boundaries rather than a confirmed design advantage.

The paper makes three empirical contributions and one external-validation protocol contribution. Empirically, it defines critical bridge leakage as a turn-level reasoning-opportunity preservation construct, provides a bridge-aware benchmark framework for competitive-programming tutoring, and reports how offline tutoring designs trade off helpfulness, bridge preservation, and learner burden across both the original 50-case evidence package and the expanded 100-case closeout. As a protocol contribution, it specifies how real AIChat logs can later be used for privacy-gated ecological-validity and rubric-transfer checks. The empirical claims are benchmark/evaluation claims: CP-MissingBridgeBench reveals strong baselines and design failure modes; it does not claim that Bridge Contract, Guard, or Repair is a deployed or universally better tutoring system.

1.1 Scope And Distinction From Nearby Work

This focus separates CP-MissingBridgeBench from two nearby lines of work. Answer-leakage robustness asks whether tutors reveal final answers or complete solutions, often under adversarial student behavior. DBox-style work studies LLM-supported co-decomposition as a learning scaffold. Our target is narrower: ordinary turn-level tutoring where the response may prematurely complete the learner’s current intermediate bridge.

This is a tutoring-specific reasoning-opportunity framing, not a general AI-safety benchmark. CP-MissingBridgeBench does not propose a DBox-style decomposition system and does not evaluate the deployed DBox system. It evaluates case-specific tutor responses, including a DBox-inspired offline tutoring design used as a strong baseline, to ask whether each response preserves the learner’s current missing reasoning bridge.

2 Related Work

2.1 Artificial Intelligence Tutoring And Intelligent Tutoring Systems For Programming

Programming tutors are often evaluated by the quality of the learning support they provide, not only by answer correctness. Work on intelligent tutoring and programming education foregrounds adaptivity, learner experience, feedback usefulness, and evaluation in authentic or semi-authentic learning settings [Chrysafiadi et al., 2023, Wang et al., 2023, Gong et al., 2025, Güner and Er, 2025]. This framing is important for CP-MissingBridgeBench because a competitive-programming tutor response can be technically correct while still being poorly timed for the learner’s current reasoning state.

What remains less visible in system-level tutoring evaluations is the moment at which a single response completes the learner’s next reasoning step. CP-MissingBridgeBench therefore moves the unit of analysis to the dialogue turn: the question is whether a response preserves a case-specific missing bridge while still giving help the learner can use.

2.2 Large Language Model Feedback And Scaffolding

Generative feedback is attractive in programming education because it can be immediate, fluent, and tailored to a learner’s expressed difficulty. Prior work on LLM feedback and scaffolding highlights this promise across explanation, debugging, decomposition, and teacher-facing feedback support [Gong et al., 2025, Guo and Wang, 2024, Ma et al., 2025]. The same fluency, however, can make a response feel helpful precisely when it has already supplied the idea the learner needed to form.

This distinction places CP-MissingBridgeBench between feedback quality and feedback appropriateness. In the binary-search vignette, asking the student to articulate what must become true as a candidate value changes can be a scaffold; giving the predicate itself can be leakage. The benchmark evaluates that boundary directly rather than treating completeness as the default mark of helpfulness.

2.3 Rubric-Based Expert Evaluation

Open-ended educational artifacts need explicit criteria if human judgments are to be interpretable. Rubric-based and expert-evaluation work motivates making the object of judgment visible before scoring begins [González-Mujico, 2024, Li et al., 2025, Maurya et al., 2025]. CP-MissingBridgeBench follows this approach by fixing the missing bridge, forbidden content, acceptable reveal, and expected student next action for each case.

The case-specific design is necessary because bridge leakage is relational. A phrase that is an acceptable hint after the student has named the key predicate may be a premature reveal before that point. Reviewer A/B review and targeted high-priority adjudication are therefore used as expert reference views with sensitivity reporting, not as a single final gold label.

2.4 Answer Leakage And Tutor Robustness

Adjacent work on tutor robustness focuses on whether students can induce tutors to reveal final answers or complete solutions, often under adversarial interaction patterns [Zhao et al., 2026]. That problem is important, but it is not the same as the ordinary help-seeking situation studied here. A student can ask a legitimate question, and a tutor can still cross the pedagogical boundary by giving the next intermediate reasoning step too early.

This leaves a middle layer under-specified: the response may avoid final-answer disclosure while still giving away the bridge that makes the solution possible. CP-MissingBridgeBench targets that intermediate layer by evaluating critical bridge leakage in ordinary competitive-programming tutoring turns.

2.5 Algorithmic-Programming Scaffolding And DBox

Algorithmic-programming scaffolding work is close to this paper because decomposition is central to learning how to solve programming problems. DBox-style co-decomposition work is especially relevant as a comparison point: it treats learner-LLM interaction as a way to structure algorithmic problem solving [Ma et al., 2025, Maurya et al., 2025]. In this paper, DBox-inspired decomposition is used as a strong offline single-turn baseline.

The comparison is deliberately limited. CP-MissingBridgeBench does not reproduce or evaluate the full deployed DBox system. It asks a narrower evaluation question: when a response takes a decomposition-oriented form, does it preserve the learner’s current missing bridge under case-specific human review?

2.6 Large Language Model Graders And Calibration

AI-supported assessment and LLM graders can help scale review, triage, or low-risk feedback workflows, but educational judgment remains especially important when the construct is contextual and high consequence [Atasoy and Arani, 2025, Li et al., 2025, Maurya et al., 2025]. Critical bridge leakage is one such construct because it depends on the problem, prior dialogue, and the learner’s demonstrated state.

For that reason, CP-MissingBridgeBench treats DeepSeek-backed grading as calibration rather than adjudication. The relevant question is not whether a grader can produce a plausible score, but whether it can catch high-risk bridge leakage; the current evidence keeps that role auxiliary to human review.

3 Methods / Evaluation

3.1 Task Definition: Critical Bridge Leakage

Critical bridge leakage is not the same as final-answer leakage. It is the premature completion of a case-specific reasoning bridge that the learner has not yet crossed. In a competitive-programming tutoring turn, that bridge may be a dynamic-programming state meaning, a binary-search predicate, an invariant, an aggregation argument, a data-structure operation, or the debugging evidence needed to locate a fault.

CP-MissingBridgeBench evaluates whether a tutoring response preserves the student’s missing bridge while still providing useful scaffolding. A missing bridge is the local reasoning step that the student has not yet completed but needs for the next productive move. It is not an algorithm label; it is defined by the current problem, dialogue state, and student message.

As a running example, suppose the student has already suspected binary search but has not yet formulated the feasibility predicate, often written as $\text{check}(x)$. The missing bridge is not "binary search" itself. It is the monotonic predicate: what candidate value x means, what property should be tested, and why the test becomes easier or harder as x changes. Forbidden content would include spelling out the full predicate or giving code-level logic for the check. Acceptable reveal might include asking the student to define what x stands for or to compare two candidate values. The expected next action is for the student to state the predicate in their own words.

Table 1 is a conceptual illustration only; it introduces the evaluation construct and does not add experimental cases or results.

Table 1 Conceptual examples of critical-bridge leakage.

student state	missing bridge	acceptable reveal	critical leakage
The student suspects binary search but cannot define the feasibility predicate $\text{check}(x)$.	The monotonic predicate and the meaning of candidate value x .	Ask what x represents, what should become easier or harder as x changes, and what property the predicate must test.	Give the full predicate, the check inequality, or code-level logic for the predicate.
The student has a recurrence intuition but cannot name the dynamic-programming state.	The state semantics and which previous state contributes to the transition.	Ask what each dimension records and which earlier choice must be known before the transition.	State the exact dynamic-programming meaning and transition formula the student has not yet derived.
The student knows a tree-path update needs marking but not how tree-difference marking works.	The endpoint, lowest common ancestor (LCA), and cancellation rule that makes path contributions aggregate correctly.	Ask which nodes should receive positive and negative marks and where the path contribution should stop.	Give the exact endpoint/LCA marking rule or the full contribution/cancellation pattern.

Critical bridge leakage occurs when the tutor supplies that local reasoning step too early. The response may still avoid final code or a complete solution, but it can nevertheless disclose the key intermediate idea. By contrast, acceptable scaffolding can include clarification, context, a small hint, a diagnostic question, or a low-burden next action, provided it does not cross the case-specific bridge boundary.

For error analysis, each failure is described along three review dimensions: the learner reasoning focus at stake, the form of premature disclosure, and the visible task or dialogue cue that supports the judgment. These dimensions make the coding auditable without turning the benchmark into a universal account of competitive-programming tutoring.

3.2 Bridge-Aware Evaluation Framework

The benchmark is organized around a bridge-aware evaluation pipeline. A bridge-identification step specifies the learner’s current missing bridge, the forbidden content that would prematurely complete that bridge, the acceptable reveal boundary, and the expected next student action. A tutoring design then generates a bounded scaffold that aims to help the learner move forward without taking over the next reasoning step.

A leakage-checking step evaluates whether the response prematurely discloses the missing bridge. This check targets intermediate reasoning-step disclosure rather than only final-answer or complete-solution leakage. When a response is over-revealing, a Repair stage can rewrite it into a more bounded scaffold; this may reduce leakage risk but can also increase learner burden if the repaired response becomes too vague or too demanding.

Human coach review remains the final evaluation authority in the current paper. Bridge Judge, Leakage Judge, and Repair components are used as offline evaluation and stress-test instruments, while LLM-based judges remain auxiliary rather than replacements for coach review. The framework is evaluated here in a fixed offline benchmark. It is not a deployed online system, and real AIChat material is treated only as an ecological-validation and replay-planning layer.

3.3 Sample Adequacy And Evidence Boundaries

CP-MissingBridgeBench is designed as a high-resolution diagnostic benchmark rather than a population-level prevalence estimate. The 50 reviewed dialogue-state cases are costly units: each case requires problem

context, recent dialogue, a student-state interpretation, case-specific forbidden content, acceptable reveal, expected next student action, and human review across anonymized tutoring responses. This design makes the sample appropriate for studying whether offline tutoring designs preserve learner reasoning opportunities under controlled human review, but it should not be interpreted as estimating how often critical bridge leakage occurs across all competitive-programming tutoring interactions.

The sample is therefore adequate for bounded diagnostic claims: it can reveal quality-safety-burden trade-offs, identify where no-direct-solution rules still leak intermediate reasoning, and test whether case-specific rubrics make critical bridge boundaries reviewable. It is not adequate for claims about population prevalence, full competitive-programming tutoring coverage, deployed AIChat advantage, or long-term learning outcomes. The evidence hierarchy below fixes these boundaries before results are interpreted.

Table 2 Evidence hierarchy and reporting boundaries.

evidence layer	unit and count	role in paper	boundary
Fixed offline benchmark	50 reviewed dialogue-state cases; 7 anonymized offline tutoring designs; 350 response-level outputs	Main human-review evidence package	Offline diagnostic benchmark, not online deployment evidence
Headline scaffold analysis	31 primary scaffold cases; 217 response-level reviews	Primary scaffold-quality, leakage-control, and learner-burden comparison	217 reviews are same-case tutoring-design outputs, not separate case-level samples
Remaining subsets and supporting checks	19 supporting cases; 133 response-level reviews; paired uncertainty, Repair stress, DBox+Repair add-on, LLM-grader calibration	Sensitivity, stress, fairness, and calibration evidence	Not mixed into headline scaffold ranking
Expanded v2 benchmark closeout	100 case-level tutoring situations; 7 fixed offline tutoring designs; 700 response-level reviews	Larger aggregate benchmark/evaluation closeout	Reported as v2 evidence; does not recompute dialogue-state v3 main tables or establish deployment effects
Online AIChat corpus and candidate screening	1156 message rows; 87 sessions; 578 paired turns; 137 substantial candidate turns; 30 selected pilot cases	Source-corpus and ecological-validity planning layer	Aggregate/process status only while consent/reporting gates remain pending
Expanded real-log-grounded closeout	100 context-sufficient real-log-grounded cases; 700 offline response-level reviews	Larger aggregate benchmark/evaluation layer	Offline and aggregate-only; not online deployment or learning-outcome evidence

The offline benchmark covers multiple learner reasoning foci and evaluation targets without treating them as one homogeneous population. The table below summarizes coverage from existing dialogue-state evaluation case metadata; it does not add cases, change subsets, or change any reported numbers.

Table 3 Benchmark coverage by subset and learner reasoning focus.

subset	cases	learner reasoning focus in case metadata	expected tutor moves	context readiness / case type	paper role
clarification-safety subset	10	state representation (6), transition / recurrence source (4)	Ask for clarification	no recent dialogue, vague question, short context with problem metadata	clarification and non-hallucination safety
primary scaffold subset	31	transition (1), predicate check (5), boundary/order (3), modeling (2), aggregation/contribution (4), data-structure operation (5), correctness/invariant (5), implementation boundary (4), debugging evidence (2)	Continue prior scaffold, Micro-scaffold	recent dialogue ends with assistant; problem context present; several code/pronoun-dependent cases	primary scaffold analysis
cautionary scaffold subset	5	boundary/order (2), modeling relation (3)	Micro-scaffold	scaffold-like cases retained with caution	sensitivity / appendix
policy-redirection subset	4	debugging evidence (1), policy request (3)	Safe redirection	direct-answer/code or policy-risk redirection cases	policy-safety appendix

This coverage table supports diagnostic breadth across learner reasoning foci and visible task cues, but it does not claim exhaustive category coverage. The benchmark is strongest when read as a case-specific human-review instrument: it asks whether each response preserves the learner’s current missing bridge, not whether the sample represents all possible competitive-programming tutoring interactions.

3.4 Unit Of Analysis

The unit of analysis is fixed separately for each evidence layer. The 50 dialogue-state evaluation items are reviewed case-level tutoring situations, not a set of student participants. The 350 outputs are response-level outputs generated by applying 7 anonymized offline tutoring designs to the 50 cases; they are not independent case-level samples. The 217 reviews in the headline scaffold subset are response-level reviews nested within 31 primary scaffold cases, not independent student samples.

The real-student AIChat layers use different units. The expanded v2 closeout consists of 100 real-log-grounded case-level tutoring situations and 700 offline response-level reviews, not a student-level learning sample. The generated responses are offline counterfactual tutoring-design outputs and were not shown to

students. Real trajectory subsets, if used in future work, would be short-horizon context checks rather than evidence of learning or long-term achievement gains.

Feasibility dry-run and focus-review workflows were used to check whether the coach-facing form and adjudication process could be executed on real AIChat cases. Because the consent/reporting gate remains pending, these workflows are reported only as process status and not as case-level evidence.

3.5 Dialogue-State Cases And Evaluation Subsets

Dialogue-state evaluation contains 50 reviewed cases. Each case keeps the problem context, recent dialogue, student message, and case-specific rubric together, so the response is judged against the actual tutoring situation rather than against an isolated problem statement. The running binary-search example illustrates why this matters: the same sentence about monotonicity may be a useful hint after the student has named the predicate, but a leakage case before that bridge has been crossed.

The dataset is organized at two levels: 50 case-level tutoring situations and 350 response-level outputs. Since each case is evaluated under 7 anonymized tutoring designs, the 31-case primary scaffold subset yields 217 tutor responses for the primary scaffold analysis.

The case-level tutoring situation is the primary unit for subset assignment, while response-level outputs are generated by applying the 7 anonymized tutoring designs to each case. Tutoring-design differences are therefore interpreted with same-case paired comparisons rather than as separate case-level samples.

Table 4 Dialogue-state evaluation subsets and response-level counts.

subset	cases	responses	evaluation target	paper use
primary scaffold subset	31	217	ordinary scaffold quality, leakage control, and student burden	Main scaffold-quality, leakage-control, and learner-burden comparison
cautionary scaffold subset	5	35	scaffold-like cases retained with caution	sensitivity / appendix
clarification-safety subset	10	70	insufficient-context clarification and non-hallucination	Clarification safety and non-hallucination under insufficient context
policy-redirection subset	4	28	direct-answer/code request redirection	Safe redirection for direct answer or code requests
all cases	50	350	full reviewed corpus across heterogeneous targets	sensitivity / appendix only

The primary analysis uses only primary scaffold subset. The other subsets and the all-50 aggregate are treated as sensitivity or appendix evidence. This separation avoids mixing ordinary scaffolding, clarification safety, and direct-answer policy handling into one headline number.

The full 50-case corpus is not discarded. Non-primary subsets are reported separately because they test different evaluation targets.

3.6 Offline Human-Review Tutoring Designs

The main human review compares seven anonymized offline tutoring designs, producing 350 candidate responses. Reviewers evaluated the responses blind to design names. After unblinding, the designs are interpreted as follows:

Table 5 Offline tutoring designs and interpretation boundaries.

tutoring design	role	interpretation boundary
Prompt-only	strong prompt-only baseline	Strong prompt-only baseline without a case-specific Bridge Contract.
No-direct-solution help	no-direct-solution programming-help baseline	Tests whether a no-direct-code / no-complete-solution rule is sufficient to prevent critical bridge leakage.
DBox-inspired	literature-inspired decomposition baseline	DBox-inspired single-turn decomposition scaffold; not a faithful DBox reproduction.
DBox-inspired + Guard	DBox-inspired guard-checked variant	Guard-only mainly provides a runtime leakage signal in the main experiment and does not rewrite the final response.
Bridge-guided DBox-style + Guard	bridge-guided decomposition variant	Uses bridge signals to guide a DBox-style response while remaining a guard-checked tutoring design.
Bridge Contract + Guard	bridge-aware guard-checked design	Uses a compact Bridge Contract and guard signal; Guard-only is not a rewrite design.
Bridge Contract + Guard/Repair	bridge-aware guard-and-repair design	The student-visible response can come from the Repair stage; main-experiment means support a design-level trend but do not by themselves prove a causal Repair effect.

These are offline evaluation designs, not online AIChat configurations and not evidence of active-mode deployment. They are used to study tutoring-response behavior under controlled review settings. The DBox-inspired rows should be read as baseline designs in this offline review setting, not as a claim about the full DBox system.

3.7 Case-Specific Rubric And Blind Review

Each case is scored against a rubric fixed before review. The rubric includes success criteria, forbidden content, critical bridge boundary, acceptable reveal, and expected next student action. In the running example, the rubric would separate a legitimate prompt such as "what must become true as x increases?" from a forbidden response that gives the full check predicate. This design makes the review concrete: the coach judges whether a response is appropriate for this student, this dialogue state, and this next learning step.

Reviewer A and Reviewer B each reviewed all 350 responses. For every response, they saw the problem context, rubric, recent dialogue, student message, and candidate reply, then assigned main metrics, diagnostic

labels, and reliability fields. Forced notes were required for major / answer leakage, a reviewer decision not to show the response to a student, low overall-quality scores, first or last rank within a case, low confidence, and samples needing discussion.

Reviewer A, Reviewer B, and targeted high-priority adjudication are expert reference or adjudicated sensitivity views. None is treated as a single definitive label. Disagreement is part of the evidence: it shows where pedagogical judgment is sensitive and where the paper must report uncertainty rather than a single settled label.

3.8 Metrics

The main metrics are:

Table 6 Human-review metrics and reader-facing labels.

reader-facing metric	review form item	role
Overall tutoring quality	overall tutoring quality	1-5 overall judgment of the response as tutoring support
Student-ready judgment	student-ready judgment	whether the reviewer would be willing to show the response to a student
Safe-ready judgment	safe-ready judgment	safety and usability threshold
Critical-leakage severity	critical-leakage severity	no leakage / minor bridge leakage / major bridge leakage / answer leakage
Scaffold sufficiency	scaffold sufficiency	guards against responses that are safe but not helpful
Learner response burden	learner response burden	input and reasoning burden required from the student in the next turn

Diagnostic metrics are used to explain failures and calibrate automatic graders. They are not used as the sole basis for design ranking. Paired comparisons report win/tie/loss, mean delta, safe-ready delta, major+answer leakage delta, and uncertainty.

3.9 Analysis Hierarchy And Evidence Manifest

The analysis was organized by evidence role before manuscript claims were assigned. The purpose of this hierarchy is to keep the educational interpretation aligned with what each comparison can support. The 31-case primary scaffold subset provides the primary scaffold evaluation. Rater-specific views, supporting subsets, and all-50 summaries are used to examine sensitivity. Same-case comparisons are used to describe uncertainty around design differences. Repair original/repared comparisons and DBox+Repair review are kept as supporting checks rather than promoted into headline evidence.

Table 7 Analysis hierarchy and evidence roles.

evidence role	use
main result	primary scaffold subset under the targeted-adjudication plus Reviewer A reporting view
sensitivity	Reviewer A only, Reviewer B only, targeted adjudication + Reviewer A, targeted adjudication + Reviewer B, all-case appendix
subset analysis	primary scaffold, cautionary scaffold, clarification safety, and policy redirection reported separately
paired uncertainty	same-case tutoring-design comparisons with win/tie/loss (W/T/L), mean delta, bootstrap confidence interval (CI), and permutation test
stress test	Repair same-candidate stress test
fairness sensitivity	DBox+Repair 20-case targeted add-on
calibration	DeepSeek LLM-grader calibration as auxiliary grader evaluation

The evidence manifest and reproduction scripts provide the audit trail for the reported tables: the evidence manifest and table-verification scripts. This traceability is methodological support, not a separate result. Sensitivity, stress, and calibration results are not promoted to main results.

3.10 Real-Log-Grounded V2 Closeout And External-Validity Boundary

The original dialogue-state v3 results come from the fixed offline benchmark; real AIChat material is handled as a separate evidence layer with its own reporting boundary. This pathway does not change or recompute the original dialogue-state v3 main tables. The expanded v2 closeout uses 100 real-log-grounded case-level tutoring situations and 7 fixed offline tutoring designs, producing 700 response-level reviews. It is reported as aggregate benchmark/evaluation evidence, not as online AIChat performance evidence.

Feasibility dry-run and focus-review workflows were used to check whether the coach-facing form and adjudication process could be executed on real AIChat cases. Because the consent/reporting gate remains pending, case-level real-student materials remain non-public. Public reporting is limited to aggregate counts, review integrity status, design-level results, and evidence-role boundaries; student text, full code, full observed responses, coach notes, row-level identifiers, and individual case labels remain restricted.

The observed current AIChat response is treated as an observed current-system response already shown to the student. It is not one of the 7 offline tutoring designs, not a control arm, not a deployment comparison arm, and not a Repair output. The 100-case closeout compares only offline generated responses under fixed tutoring designs; no generated response was shown to students.

The expanded v2 closeout is a same-starting-state offline comparison of fixed tutoring designs on real-log-grounded cases. It remains offline: responses were generated for review, hidden from students, randomized for coach evaluation, and reported only as aggregate benchmark/evaluation evidence. These results are not merged into the original dialogue-state v3 primary tables.

To avoid conflating observation and intervention, the observed current AIChat response is treated as a current-system record rather than as an experimental comparator. The v2 closeout is used to examine whether the benchmark and rubric can scale to a larger real-log-grounded evaluation setting. It does not validate active deployment, deployed-system advantage, or learning outcomes.

4 Results

The results are organized around the original dialogue-state benchmark, the expanded v2 closeout, and supporting stress/calibration evidence. The original headline result uses 31 case-level situations and 217 response-level reviews from the primary scaffold subset under targeted-adjudication plus Reviewer A. The expanded v2 closeout uses 100 case-level situations and 700 response-level reviews as a separate aggregate benchmark/evaluation layer. Repair original/repared comparisons are stress evidence; DBox+Repair is a targeted fairness sensitivity add-on; DeepSeek LLM-grader calibration is auxiliary and remains outside the human adjudication role.

4.1 RQ1: How Consistently Can Coaches Judge Critical-Bridge Leakage?

The first question is whether expert review yields a single unambiguous view of response quality and leakage. Both coaches reviewed all 350 responses from the 50 reviewed cases and 7 anonymized tutoring designs. Overall exact agreement between Reviewer A and Reviewer B is 0.2829, while within-1 agreement is 0.8429. Leakage-label exact agreement is 0.6714. Critical-binary exact agreement is 0.9029, but kappa is 0.2511.

The targeted adjudication set contains 60 high-priority disagreements: 29 followed Reviewer A, 9 followed Reviewer B, and 22 received a new adjudicated label. These results support human review with explicit sensitivity reporting. They do not justify treating Reviewer A, Reviewer B, or targeted high-priority adjudication as a single settled reference. Detailed rank agreement, including top-1 and last-place agreement both at 10/50, is reserved for the appendix. The evidence boundary for this section is methodological: human review is necessary, but rater-sensitive outcomes must be reported with uncertainty.

4.2 RQ2: How Do Tutoring Designs Trade Off Helpfulness, Bridge Preservation, And Learner Burden?

The headline evaluation compares the 7 anonymized tutoring designs on the 31-case primary scaffold subset under the `targeted-adjudication plus Reviewer A` view. Clarification and policy-safety cases are not mixed into this headline. Table 8 uses reader-facing tutoring-design labels, matching the design descriptions in Table 5.

Table 8 Primary scaffold-subset results under the primary review view.

tutoring design	overall	student-ready	safe-ready	major+answer leakage
Prompt-only	3.032	7/31	7/31	9
No-direct-solution help	3.710	19/31	19/31	2
DBox-inspired	3.774	21/31	21/31	2
DBox-inspired + Guard	3.774	23/31	23/31	2
Bridge-guided DBox-style + Guard	3.742	20/31	20/31	2
Bridge Contract + Guard	4.000	23/31	23/31	0
Bridge Contract + Guard/Repair	4.065	22/31	22/31	0

In this subset, Bridge Contract + Guard/Repair has the highest overall score, and both Bridge Contract

compact variants have 0 major+answer leakage. At the same time, DBox-inspired decomposition remains a strong baseline: DBox-inspired + Guard is close to the Bridge Contract compact variants on student-ready and safe-ready counts. The table therefore supports a quality-safety-burden trade-off rather than a single-design victory.

The contrast between No-direct-solution help and Prompt-only also supports the educational problem statement. A no-direct-code / no-direct-solution baseline is stronger than prompt-only, but it still has 2 major+answer leakage cases in the primary subset. Design-level ranks and non-primary subset tables are reserved for the appendix. The evidence boundary is that this table does not establish broad advantage of Bridge Contract over all baselines and does not validate online deployment; all compared outputs are offline tutoring designs.

4.3 Expanded V2 Closeout: What Changes In A 100-Case Benchmark Review?

The expanded v2 closeout asks whether the same evaluation framework still supports the same design story on a larger case set. It uses 100 case-level tutoring situations and the same 7 fixed offline tutoring designs, yielding 700 response-level reviews. The integrity audit passes at the aggregate level: 100 cases, 700 reviewed rows, 700 unique anonymized response identifiers, 100 rows per design, and no missing required review fields. The v2 closeout is not merged into the original dialogue-state v3 main tables; it is reported as a larger benchmark/evaluation layer.

Table 9 Expanded v2 100-case / 700-response closeout results.

tutoring design	overall	student-ready	safe-ready	major+answer leakage	burden low/medium/high
Prompt-only	3.91	64/100	44/100	13	37/55/8
No-direct-solution help	3.57	50/100	37/100	12	48/50/2
DBox-inspired	4.08	64/100	51/100	4	66/32/2
DBox-inspired + Guard	4.23	72/100	54/100	1	69/30/1
Bridge-guided DBox-style + Guard	3.70	45/100	39/100	5	57/42/1
Bridge Contract + Guard	3.82	57/100	51/100	6	63/35/2
Bridge Contract + Guard/Repair	3.75	53/100	45/100	9	58/37/5

The expanded closeout changes the interpretation of the paper. In the 31-case primary scaffold subset, Bridge Contract compact + Guard/Repair showed a favorable but uncertain trend. In the 100-case closeout, DBox-inspired + Guard is clearly the strongest aggregate design: it has the highest overall mean (4.23), the highest student-ready count (72/100), the highest safe-ready count (54/100), and the lowest major+answer leakage count (1/100). Bridge Contract + Guard/Repair is lower on overall quality (3.75), student-ready count (53/100), safe-ready count (45/100), and high-severity leakage control (9/100 major+answer leakage).

Table 10 Key paired comparisons in the expanded v2 closeout.

paired comparison	Δ overall	W/T/L	bootstrap 95% CI
Bridge Contract + Guard – DBox-inspired + Guard	-0.41	18/45/37	[-0.63, -0.19]
Bridge-guided DBox-style + Guard – DBox-inspired + Guard	-0.53	14/45/41	[-0.76, -0.32]
Bridge Contract + Guard/Repair – DBox-inspired + Guard	-0.48	22/37/41	[-0.74, -0.24]
Bridge Contract + Guard/Repair – Bridge Contract + Guard	-0.07	24/45/31	[-0.36, 0.19]
DBox-inspired + Guard – DBox-inspired	+0.15	30/47/23	[-0.04, +0.35]

These paired comparisons rule out a single-design-win reading for the current Bridge Contract / Repair pipeline. The benchmark still supports the construct: no-direct-solution prompting has 12 major+answer leakage cases in the 100-case closeout, so final-answer avoidance alone does not solve premature reasoning-step disclosure. But the strongest design in the expanded review is the decomposition-oriented DBox-inspired + Guard baseline. The scientific contribution of CP-MissingBridgeBench is therefore the evaluation framework and the design diagnosis it makes possible, not a claim that Bridge Contract or Repair is already the best tutoring method.

The expanded source set is also not balanced across all reasoning foci. It is implementation/debugging heavy: implementation and debugging help-seeking account for 85/100 cases, and the largest reasoning-focus groups are implementation bridge (39/100) and debugging bridge (34/100). This distribution is useful because it reflects a common real-tutoring pressure point, but it limits any claim about balanced coverage across all competitive-programming reasoning families.

4.4 RQ3: How Much Uncertainty Remains In Same-Case Comparisons?

Same-case paired comparisons ask whether observed differences remain persuasive when tutoring designs are compared on the same cases. For Bridge Contract + Guard/Repair vs DBox-inspired + Guard, mean overall delta is +0.290, W/T/L is 14/10/7, and major+answer leakage is lower by 2 cases. The paired bootstrap 95% CI is [-0.097, +0.645], and the paired permutation p-value is 0.2016.

For Bridge Contract + Guard/Repair vs Bridge Contract + Guard, mean overall delta is +0.065, W/T/L is 7/18/6, and major+answer leakage is the same. By contrast, No-direct-solution help vs Prompt-only has a mean overall delta of +0.677, 95% CI [+0.258, +1.065], paired $p=0.0046$, 12 more safe-ready cases, and 7 fewer major+answer leakage cases.

These comparisons explain why the original 31-case result should be read as a bounded local pattern rather than as a method claim. They support a possible trade-off advantage for Bridge Contract compact + Guard/Repair within the primary scaffold subset, and a clearer improvement of no-direct-solution programming help over prompt-only help. The expanded 100-case closeout then supplies the stronger external check: the local Bridge Contract trend does not generalize into overall advantage over DBox-inspired + Guard. The evidence boundary is important: the paired analysis does not support a strong advantage claim for Bridge Contract compact + Guard/Repair over DBox-inspired Guard, and the main tutoring-design comparison does not establish a causal Repair effect.

4.5 RQ4: What Pedagogical Failure Patterns Does The Benchmark Surface?

The error analysis asks whether failures can be described beyond isolated algorithm scenes, such as "this is a dynamic-programming case" or "this is a binary-search case." We therefore coded each observed failure by the tutoring problem it reflected, the learner reasoning focus at stake, and the visible task cue that made the judgment possible.

The review categories include critical bridge leakage, answer/code leakage, over-complete micro-examples, shifted focus, under-scaffolding, excessive burden, context misalignment, over-safe refusal, factual/algorithmic error, and policy/direct-answer handling failure. Learner reasoning foci include representation semantics, transition/action mapping, predicate/decision semantics, ordering/dependency control, modeling relation, aggregation/contribution accounting, data-structure operation mapping, correctness/invariant reasoning, implementation boundary, debugging evidence, and policy-request handling. Dynamic-programming state, binary-search check, lazy propagation, tree difference, local code, greedy proof, and debugging trace are treated as visible task cues rather than standalone categories.

This coding scheme supports analysis at the level of reusable learner reasoning foci and forms of premature disclosure. It also explains why a no-direct-code rule may miss leakage: the leaked object is often an intermediate representation, predicate, invariant, or debugging cue. Extended coding examples and reviewer notes are reserved for the appendix. The evidence boundary is that the scheme is observed and operational for this benchmark; it is not a universal coding system and does not claim exhaustive coverage of all tutoring failures.

4.6 RQ5: What Does Repair Add, And What Burden Can It Introduce?

Repair is evaluated separately from main design means because the main experiment compares outputs from different tutoring designs. The same-candidate stress test fixes the original candidate and blinds the comparison between original response and repaired response. In the 30-pair same-candidate stress test, Repair improves leakage severity in 16/30 pairs, preserves it in 14/30, and worsens it in 0/30. Major leakage falls from 7/30 to 0/30. Overall quality rises from 3.367 to 3.633, with mean delta +0.267. The burden trade-off is burden improved / same / worsened = 2/16/12.

DBox+Repair fairness is a targeted 20-case sensitivity add-on, not a new main design. DBox-inspired + Guard/Repair has overall 3.55, safe-ready 11/20, and no/minor/major+answer leakage 14/6/0. Relative to the same-case DBox-inspired Guard subset, it modestly improves overall (+0.15, W/T/L=6/9/5) and reduces major+answer leakage from 2 to 0. Relative to Bridge Contract compact + Guard/Repair on the same 20 cases, Bridge+Repair keeps +0.50 overall, W/T/L=12/5/3, and safe-ready +4.

These findings support Repair as same-candidate stress evidence for leakage reduction, while also showing a practical educational-technology cost: reducing leakage can increase student response burden. Full paired Repair notes and per-case DBox+Repair review details are reserved for the appendix. The evidence boundary is that DBox+Repair is not a full main design, the analysis does not prove Bridge advantage over every repair-enabled baseline, and Repair causality is not inferred from the main design means.

4.7 RQ6: Can LLM Graders Support, But Not Replace, Coach Review?

LLM-grader calibration asks whether automatic graders can provide reliable auxiliary signals for critical bridge leakage. The calibration reported here uses the DeepSeek flash model variant with thinking disabled, aligned with the offline judge stack. Against the targeted-adjudication reference, the DeepSeek case-specific rubric judge improves some auxiliary metrics over the generic rubric: leakage-severity accuracy is 0.617 vs 0.583, student-ready agreement is 0.467 vs 0.433, and safe-ready agreement is 0.533 vs 0.400.

However, both generic and case-specific DeepSeek graders have critical recall 0 and major leakage false-negative rate 1.000. They fail to retrieve rows that human adjudication marks as major bridge leakage

or answer leakage. Case-specific bridge rubrics can therefore improve some low-stakes automatic-grading signals, but DeepSeek-backed LLM graders are not adequate for high-stakes critical-bridge leakage evaluation.

Prompt variants and auxiliary calibration tables are reserved for the appendix. Because the generation, guard, and grader stack share the same DeepSeek-family backend, this calibration should be read as an within-stack auxiliary check rather than evidence that LLM grading transfers across model families. Human review and adjudication remain necessary.

5 Discussion

5.1 The Evaluation Framing Is Tutoring-Specific

The framing in this paper is deliberately narrow. Critical bridge leakage is a tutoring-specific failure of reasoning-opportunity preservation: it concerns whether a tutor preserves the work a learner is ready to do in a competitive-programming turn. It is not a broad account of safety across all AI-supported educational settings, and it is not a benchmark for adversarial attacks. This scope matters because critical bridge leakage can occur in ordinary help-seeking dialogue, not only when a student tries to extract an answer.

5.2 No-Direct-Answer Rules Are Insufficient

Direct-answer avoidance is not enough for turn-level tutoring safety. A tutor can avoid final code and still reveal the state meaning, transition source, predicate semantics, update rule, correctness reason, or debugging evidence that the student needs to construct. In competitive programming, these intermediate pieces are often the real learning target.

This matters for system design. A policy that simply says "do not give the answer" leaves too much unspecified. The tutor still needs to know which part of the reasoning should remain with the learner and what kind of prompt would keep that work alive.

5.3 Human Review Is Necessary For Case-Specific Bridge Boundaries

Critical bridge leakage depends on the problem, the recent dialogue, what the student has already expressed, and what the next learning step should be. Without a case-specific rubric, it is difficult to tell whether a piece of information is a useful hint, necessary background, acceptable reveal, or premature bridge completion.

The review protocol makes these boundaries explicit at the case level. Reviewer disagreement is not merely noise; it is evidence that some judgments are pedagogically sensitive. For that reason, the paper reports reliability, targeted adjudication, subset analysis, and sensitivity views instead of presenting one reviewer or targeted high-priority adjudication as a single definitive reference.

5.4 Repair Is Promising But Burden-Bearing

Guard-only variants are guard-instrumented / guard-checked variants. They expose leakage risk, but the rewrite signal does not replace the final response text except under block fallback. Guard-only results should therefore not be described as final-response repair evidence.

Repair is more directly evaluated in the same-candidate stress test. The test shows reduced leakage severity, but it also shows cases where student burden worsens. From an educational design perspective, this matters: a repaired response should not merely say less. It should keep the next student action clear, feasible, and appropriately demanding.

5.5 Automatic Graders Remain Auxiliary

LLM graders may help with scale, triage, or low-risk signals, but the dialogue-state evaluation calibration shows that they cannot currently adjudicate high-risk critical bridge leakage alone. In the targeted-adjudication reference, critical recall is 0 and the major leakage false-negative rate is 1.000. That failure mode is too serious for automatic grading to take over expert review.

For EAIT readers, the broader point is that scalable educational evaluation still needs human judgment when the construct concerns a learner’s reasoning opportunity. Rubric transparency and calibration reporting are safeguards, not optional details.

5.6 Practical Implications For Educational Technology Design

The design implication is not to make tutors withhold help. It is to make help controllable. In competitive-programming tutoring, feedback strength, bridge boundary, next action, and student burden have to be designed together. Bridge Contract-style constraints remain useful as specification tools because they force generation, review, and repair to refer to the same case-specific tutoring target; the expanded v2 closeout shows that this specification alone is not enough to beat a strong decomposition-oriented baseline.

For teachers, the benchmark does not suggest that hints should be weaker. It separates the strength of help from ownership of the next reasoning step. A good AI tutor can be specific about where to look, what to test, or what to compare, while still leaving the learner to formulate the predicate, state meaning, invariant, or debugging explanation.

Human review also plays a design role, not only an audit role. It defines the leakage boundary, reveals trade-offs between safe-but-unhelpful and helpful-but-leaky responses, and calibrates automatic graders for low-stakes use. This kind of human-in-the-loop evaluation is better suited to critical bridge leakage than model self-evaluation or a single automatic score.

5.7 Real-Student AIChat As An Aggregate Real-Log-Grounded Layer

The real-student AIChat material should be read as aggregate real-log-grounded benchmark support, not as a system-performance experiment. The observed response record documents the response that the existing online AIChat had already shown to the student. It is an observed current-system response rather than an experimental comparator. It is not one of the seven offline tutoring designs, not a baseline, not a control arm, and not a Repair output.

This distinction protects both the paper’s evidence hierarchy and the educational interpretation. The expanded v2 closeout shows that real-log-grounded student turns from our own teaching system can be converted into 100 reviewable case-level tutoring situations and 700 offline response-level reviews. It cannot show that the deployed AIChat performs better, that active mode is validated, or that student learning outcomes improve.

Any future shadow-mode or risk-triggered deployment should be reported as a separate deployment-readiness study rather than merged into the dialogue-state evaluation primary evidence. The current v2 evidence remains offline and aggregate-only.

5.8 Limitations

This study has nine main limitations. First, the original headline subset is 31 cases / 217 response-level reviews and should be interpreted as bounded expert-reviewed evidence, not exhaustive competitive-programming tutoring coverage; the 217 reviews are response-level outputs nested within 31 case-level situations. Second, the original 50 cases are curated high-risk dialogue-state cases rather than a random sample of all competitive-programming tutoring interactions, so the benchmark does not estimate population

prevalence of critical bridge leakage. Third, the expanded v2 closeout improves the scale to 100 cases / 700 reviewed responses, but it is implementation/debugging heavy and should not be described as a balanced sample of all reasoning-bridge families. Fourth, student-ready, safe-ready, and rank outcomes are rater-sensitive, so sensitivity views are necessary. Fifth, targeted high-priority adjudication is not a single definitive reference; it reduces uncertainty for high-priority disagreements but does not remove all rater differences. Sixth, DBox+Repair is a targeted 20-case sensitivity review, not a full 50-case double-review design. Seventh, DeepSeek-backed LLM-grader calibration has high critical false-negative risk and same-backend coupling, so automatic graders and judges remain auxiliary rather than replacements for human coaches. Eighth, the paper evaluates offline tutoring designs and separate real AIChat aggregate evidence; it does not evaluate long-term learning outcomes, deployed AIChat advantage, online active-mode effectiveness, or population-level usage effects. Ninth, any future release of case-level real-student material would require additional privacy, consent/reporting, selection-bias, and double-review checks beyond the aggregate reporting used here.

6 Conclusion

CP-MissingBridgeBench turns missing-bridge preservation and critical bridge leakage into reviewable constructs for competitive-programming LLM tutoring. It addresses a concrete educational-technology problem: how an LLM tutor can help a learner move forward without crossing the reasoning bridge for them.

The evidence supports a benchmark/evaluation conclusion rather than a single-method conclusion. On the original 31-case primary scaffold subset, Bridge Contract compact + Guard/Repair shows a favorable but uncertain local trend. In the expanded 100-case / 700-response closeout, DBox-inspired + Guard is the strongest aggregate design, while Bridge Contract / Repair variants are lower on overall quality, student-ready counts, safe-ready counts, and high-severity leakage control. Together, these findings show why CP-MissingBridgeBench is useful: it can reveal when a plausible bridge-aware pipeline works locally, when it fails under a larger review, and which baselines remain difficult to beat.

The paper does not claim that 50 or 100 cases cover all competitive-programming tutoring or that coach labels or targeted high-priority adjudication provide one settled reference. It also does not treat guard instrumentation as rewrite evidence, infer Repair causality from main design means, or assign LLM graders the role of human adjudication. Its contribution is a reproducible bridge-aware benchmark framework for studying how LLM tutoring can support programming learners while preserving their opportunity to complete the missing bridge.

7 Ethics, Privacy, Data Availability, And AI Writing Disclosure

This manuscript uses an existing offline dialogue-state evaluation evidence package. The study does not modify the main experiment, add a tutoring design, recompute the main tables, alter online AIChat prompts, enable active mode, or change student-visible responses. The dialogue-state evaluation benchmark is therefore reported as a fixed offline human-review evaluation rather than as a deployed-system intervention.

7.1 Public Materials And Reproducibility

Materials suitable for public or reviewer-facing release include benchmark metadata, schemas, rubric definitions, aggregate tables, reproduction scripts, the evidence manifest, anonymized tutoring-design descriptions, citation files, and the manuscript source. These materials are sufficient to inspect the evaluation structure, claim boundaries, and aggregate results without exposing private student content. Any public release remains subject to venue policy, author review, and final privacy screening.

7.2 Restricted Materials

Verbatim dialogue, code excerpts, full observed response transcripts, identity fields, account identifiers, school names, phone numbers, email addresses, hash salts, reversible mappings, private workbooks, and any material that could identify a student or learning context are restricted materials. They should not be included in the manuscript, public appendices, public supplementary files, screenshots, or repository exports. If a real-student example is later needed, it should be paraphrased, anonymized, and checked against the consent/reporting gate rather than copied from the private packet.

7.3 Real-Student AIChat Pilot Boundary

The real-student AIChat pilot is a separate ecological-validity layer. Feasibility dry-run and focus-review workflows may be described only as process checks for coach-facing review and adjudication, not as reportable model-performance evidence. While the consent/reporting gate is pending, this layer may report only aggregate/process counts, schema readiness, screening flow, and field-sufficiency issues. It must not report case-level leakage labels, individual hash tables, full student messages, complete code snippets, complete observed current-system responses, coach notes, private score distributions, or identifiable examples.

7.4 Data Availability Statement

The authors plan to make public the non-sensitive materials needed to inspect the evaluation structure and reproduce aggregate tables: anonymized metadata, rubric and schema definitions, aggregate outputs, scripts, and citation files. Private educational interaction data, identity metadata, hash mappings, and local review workbooks are not publicly available. Any access to restricted material would require author approval, documented privacy review, and the relevant consent/reporting eligibility.

7.5 AI Writing Disclosure

AI writing assistance was used for drafting, editing, checklist generation, format conversion, and evidence organization. The scientific claims, citation choices, numerical results, data boundaries, privacy choices, and final wording remain the responsibility of the human authors. Before submission, the authors should complete venue-specific checks for AI writing disclosure, citation formatting, result-number verification, ethics/privacy review, and data availability wording.

A Case Inventory And Supplementary Transparency Boundary

Appendix A includes a cleaned submission-safe transparency inventory for all 50 dialogue-state cases. The inventory is not a new experiment, does not add tutoring designs, and does not change any result number. It is included to make benchmark coverage auditable at the case level while keeping the raw/private audit table restricted.

The cleaned inventory contains structural metadata only: case identifier, subset assignment, learner reasoning focus, visible task cue, expected tutor move, context-readiness summary, English learner-state paraphrase, and paper role. For readability, the inventory is split into two linked tables: Table A1 reports case roles and review focus, and Table A2 reports context readiness and learner-state paraphrases. It does not reproduce verbatim student messages, full task materials, code excerpts, full observed response transcripts, source-location details, or unresolved audit notes.

Bridge-family glossary: state-representation cases concern what a dynamic-programming state stores; transition cases concern where a recurrence comes from; predicate cases concern truth direction or boundary

update; modeling cases concern object relations; aggregation cases concern contribution placement; data-structure cases concern maintained summaries; correctness cases concern invariants or exchange objects; implementation and debugging cases concern bounded checks; policy cases concern safe redirection from direct solution-level requests.

Table A1 Cleaned 50-case transparency inventory: case roles and review focus.

case	subset	learner reasoning focus	visible task cue	expected tutor move	paper role
Case 001	clarification-safety subset	State representation	State semantics	Ask for clarification	clarification-safety appendix
Case 002	clarification-safety subset	State representation	State semantics	Ask for clarification	clarification-safety appendix
Case 003	clarification-safety subset	State representation	State semantics	Ask for clarification	clarification-safety appendix
Case 004	clarification-safety subset	State representation	State semantics	Ask for clarification	clarification-safety appendix
Case 005	clarification-safety subset	State representation	State semantics	Ask for clarification	clarification-safety appendix
Case 006	clarification-safety subset	State representation	State semantics	Ask for clarification	clarification-safety appendix
Case 007	clarification-safety subset	Transition / recurrence	Transition design	Ask for clarification	clarification-safety appendix
Case 008	clarification-safety subset	Transition / recurrence	Transition design	Ask for clarification	clarification-safety appendix
Case 009	clarification-safety subset	Transition / recurrence	Transition design	Ask for clarification	clarification-safety appendix
Case 010	clarification-safety subset	Transition / recurrence	Transition design	Ask for clarification	clarification-safety appendix
Case 011	primary scaffold subset	Transition / recurrence	Transition design	Continue prior scaffold	primary scaffold analysis
Case 012	primary scaffold subset	Predicate check	Predicate truth direction	Continue prior scaffold	primary scaffold analysis
Case 013	primary scaffold subset	Predicate check	Predicate truth direction	Continue prior scaffold	primary scaffold analysis
Case 014	primary scaffold subset	Predicate check	Predicate truth direction	Continue prior scaffold	primary scaffold analysis
Case 015	primary scaffold subset	Predicate check	Predicate truth direction	Continue prior scaffold	primary scaffold analysis
Case 016	primary scaffold subset	Predicate check	Predicate truth direction	Continue prior scaffold	primary scaffold analysis

Case 017	primary scaffold subset	Boundary / update order	Boundary update	Continue prior scaffold	primary scaffold analysis
Case 018	cautionary scaffold subset	Boundary / update order	Boundary update	Micro-scaffold	cautionary sensitivity appendix
Case 019	primary scaffold subset	Boundary / update order	Boundary update	Continue prior scaffold	primary scaffold analysis
Case 020	cautionary scaffold subset	Boundary / update order	Boundary update	Micro-scaffold	cautionary sensitivity appendix
Case 021	primary scaffold subset	Boundary / update order	Boundary update	Micro-scaffold	primary scaffold analysis
Case 022	primary scaffold subset	Modeling relation	Object-relation modeling	Micro-scaffold	primary scaffold analysis
Case 023	cautionary scaffold subset	Modeling relation	Object-relation modeling	Micro-scaffold	cautionary sensitivity appendix
Case 024	cautionary scaffold subset	Modeling relation	Object-relation modeling	Micro-scaffold	cautionary sensitivity appendix
Case 025	primary scaffold subset	Modeling relation	Object-relation modeling	Micro-scaffold	primary scaffold analysis
Case 026	cautionary scaffold subset	Modeling relation	Object-relation modeling	Micro-scaffold	cautionary sensitivity appendix
Case 027	primary scaffold subset	Aggregation / contribution	Contribution marking	Micro-scaffold	primary scaffold analysis
Case 028	primary scaffold subset	Aggregation / contribution	Contribution marking	Micro-scaffold	primary scaffold analysis
Case 029	primary scaffold subset	Aggregation / contribution	Contribution marking	Micro-scaffold	primary scaffold analysis
Case 030	primary scaffold subset	Aggregation / contribution	Contribution marking	Micro-scaffold	primary scaffold analysis
Case 031	primary scaffold subset	Data-structure operation	Operation semantics	Micro-scaffold	primary scaffold analysis

Case 032	primary scaffold subset	Data-structure operation	Operation semantics	Micro-scaffold	primary scaffold analysis
Case 033	primary scaffold subset	Data-structure operation	Operation semantics	Micro-scaffold	primary scaffold analysis
Case 034	primary scaffold subset	Data-structure operation	Operation semantics	Micro-scaffold	primary scaffold analysis
Case 035	primary scaffold subset	Data-structure operation	Operation semantics	Micro-scaffold	primary scaffold analysis
Case 036	primary scaffold subset	Correctness / invariant	Invariant reasoning	Micro-scaffold	primary scaffold analysis
Case 037	primary scaffold subset	Correctness / invariant	Invariant reasoning	Continue prior scaffold	primary scaffold analysis
Case 038	primary scaffold subset	Correctness / invariant	Invariant reasoning	Micro-scaffold	primary scaffold analysis
Case 039	primary scaffold subset	Correctness / invariant	Invariant reasoning	Micro-scaffold	primary scaffold analysis
Case 040	primary scaffold subset	Correctness / invariant	Invariant reasoning	Micro-scaffold	primary scaffold analysis
Case 041	primary scaffold subset	Implementation boundary	Implementation boundary	Continue prior scaffold	primary scaffold analysis
Case 042	primary scaffold subset	Implementation boundary	Implementation boundary	Micro-scaffold	primary scaffold analysis
Case 043	primary scaffold subset	Implementation boundary	Implementation boundary	Micro-scaffold	primary scaffold analysis
Case 044	primary scaffold subset	Implementation boundary	Implementation boundary	Micro-scaffold	primary scaffold analysis
Case 045	policy-redirection subset	Debugging evidence	Debugging evidence	Safe redirection	policy-redirection appendix
Case 046	primary scaffold subset	Debugging evidence	Debugging evidence	Micro-scaffold	primary scaffold analysis
Case 047	primary scaffold subset	Debugging evidence	Debugging evidence	Continue prior scaffold	primary scaffold analysis
Case 048	policy-redirection subset	Policy redirection	Safe response boundary	Safe redirection	policy-redirection appendix

Case 049	policy-redirection subset	Policy redirection	Safe response boundary	Safe redirection	policy- redirection appendix
Case 050	policy-redirection subset	Policy redirection	Safe response boundary	Safe redirection	policy- redirection appendix

Table A2 Cleaned 50-case transparency inventory: context readiness and learner-state paraphrase.

case	context readiness	learner-state paraphrase
Case 001	insufficient; no recent dialogue; vague; problem context available	Initial question: state meaning is unclear and clarification is expected.
Case 002	insufficient; no recent dialogue; vague; problem context available	Initial question: table-cell semantics are unclear and clarification is expected.
Case 003	insufficient; no recent dialogue; vague; problem context available	Initial question: stored quantity is unclear and clarification is expected.
Case 004	insufficient; no recent dialogue; vague; problem context available	Initial question: state dimensions are not yet grounded and clarification is expected.
Case 005	insufficient; no recent dialogue; vague; problem context available	Initial question: variables to retain in state are unclear and clarification is expected.
Case 006	insufficient; no recent dialogue; vague; problem context available	Initial question: state representation is not yet linked to recurrence and clarification is expected.
Case 007	insufficient; no recent dialogue; vague; problem context available	Initial question: transition branches are underspecified and clarification is expected.
Case 008	insufficient; no recent dialogue; vague; problem context available	Initial question: recurrence source states are unclear and clarification is expected.
Case 009	insufficient; no recent dialogue; vague; problem context available	Initial question: recurrence step is too vague for scaffold continuation.
Case 010	insufficient; no recent dialogue; vague; problem context available	Initial question: previous-state dependency is unclear and clarification is expected.
Case 011	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: transition source is partly identified but not tied to the problem action.
Case 012	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: feasibility truth direction is being checked but still depends on prior context.
Case 013	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: candidate feasibility is stated but the bridge boundary remains predicate semantics.
Case 014	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: predicate meaning is emerging but should not be completed by the tutor.

Case 015	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: truth-direction wording is tentative and depends on prior scaffold context.
Case 016	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: feasibility interpretation is plausible but still needs learner-side formulation.
Case 017	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: update direction depends on protecting old values from overwrite.
Case 018	partial; recent assistant scaffold; medium; problem context available	Caution case: update order is partly visible but the old/new value distinction remains uncertain.
Case 019	sufficient; recent assistant scaffold; prior-context dependent; problem context available	Follow-up turn: the learner recognizes overwrite risk but has not fixed the order rule.
Case 020	partial; recent assistant scaffold; medium; problem context available	Caution case: boundary update is partially understood but the overwritten quantity is unclear.
Case 021	sufficient; recent assistant scaffold; specific; problem context available	Follow-up turn: old/new value ordering and boundary movement still need a micro-scaffold.
Case 022	sufficient; recent assistant scaffold; specific; problem context available	Follow-up turn: problem objects are listed but their relations are not yet modeled.
Case 023	partial; recent assistant scaffold; medium; problem context available	Caution case: object relations are partially available but still under-specified.
Case 024	partial; recent assistant scaffold; medium; problem context available	Caution case: task entities are visible but the abstraction relation is uncertain.
Case 025	sufficient; recent assistant scaffold; specific; problem context available	Follow-up turn: objects are available, but dependency or coverage relations remain unclear.
Case 026	partial; recent assistant scaffold; medium; problem context available	Caution case: the learner suspects a reversed modeling relation and needs a bounded prompt.
Case 027	sufficient; recent assistant scaffold; specific; problem context available	Follow-up turn: local contribution is visible but mark placement is not yet derived.
Case 028	sufficient; recent assistant scaffold; specific; problem context available	Follow-up turn: path or interval effects are visible but cancellation marks are unclear.
Case 029	sufficient; recent assistant scaffold; specific; problem context available	Follow-up turn: many local effects need to be summarized without revealing the marking rule.
Case 030	sufficient; recent assistant scaffold; specific; problem context available	Follow-up turn: aggregation near shared points remains the missing contribution bridge.

Case 031	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Implementation context: maintained data-structure summary is not yet semantically grounded.
Case 032	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Implementation context: update preservation and query composition are the missing bridge.
Case 033	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Debugging context: after an update, the changed maintained summary is unclear.
Case 034	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Implementation context: query meaning should be tied to maintained summaries.
Case 035	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Debugging context: the likely fault is a maintained summary rather than surface syntax.
Case 036	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Reasoning context: a local comparison object is needed for a correctness argument.
Case 037	sufficient; recent assistant scaffold; prior-context dependent; code excerpt available; problem context available	Follow-up turn: the exchange argument depends on identifying what does not get worse.
Case 038	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Reasoning context: the invariant behind adjacent-step exchange is not yet explicit.
Case 039	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Reasoning context: a small adjacent exchange should support the learner proof attempt.
Case 040	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Reasoning context: the proof target is the metric preserved or improved by exchange.
Case 041	sufficient; recent assistant scaffold; prior-context dependent; code excerpt available; problem context available	Implementation context: prior scaffold and code context make this a bounded debugging continuation.
Case 042	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Debugging context: a small intermediate value should be checked before any rewrite.

Case 043	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Implementation context: a single-symbol or boundary case should anchor the counting logic.
Case 044	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Implementation context: character-grid mapping needs a bounded positional check.
Case 045	partial; recent assistant scaffold; direct-solution pressure; code excerpt available; problem context available	Policy-safety case: the debugging request is framed as solution- or code-level help, so the tutor should redirect toward a bounded diagnostic step.
Case 046	sufficient; recent assistant scaffold; specific; code excerpt available; problem context available	Debugging context: a minimal checkpoint should narrow the failing state variable.
Case 047	sufficient; recent assistant scaffold; prior-context dependent; code excerpt available; problem context available	Follow-up debugging turn: a small malformed input or parsing check should be selected without giving a full fix.
Case 048	partial; recent assistant scaffold; direct-solution pressure; problem context available	Policy-safety case: direct-solution pressure should be redirected to a safe next step.
Case 049	partial; recent assistant scaffold; direct-solution pressure; problem context available	Policy-safety case: time-pressure for complete solution or code should trigger redirection.
Case 050	partial; recent assistant scaffold; direct-solution pressure; problem context available	Policy-safety case: full-solution request should be redirected toward the smallest useful learner action.

B Real-Student AIChat External-Validation Boundary

The real-student online AIChat material is included only as an ecological-validity and real-log-grounded benchmark layer. The data source is our own online AIChat / teaching system. The pilot, v2 closeout, and any replay protocols do not use Luogu discussion areas, public forums, third-party Q&A sites, social media, or any other public community data.

The online data funnel is reported at three levels. The first level summarizes the online log corpus: 1156 raw AIChat message rows, 87 sessions, and 578 paired user-assistant turns. The second level screens 137 substantial candidate turns across 59 candidate sessions; this is a lightweight candidate screening pool, not a deep annotation sample. The third level contains 30 selected pilot cases covering 11 hashed students and 15 hashed problems; these cases are candidates for full rubric annotation after privacy and consent/reporting gates.

Feasibility dry-run and focus-review workflows were used to check whether coach-facing review and adjudication could be executed on real AIChat cases. The expanded v2 closeout now reports aggregate 100-case / 700-response review results, but the reporting gate still prevents release of case-level real-student materials. Case-level labels, leakage distributions tied to individual cases, student text, code, full AIChat replies, coach notes, and identifiable examples are not reported in the manuscript.

For private coach review, missing problem-context summaries were supplemented from author-provided or locally available problem-context materials. Full problem statements and private review packets are restricted materials and are not released.

The expanded v2 closeout selects 100 context-sufficient real-log-grounded cases from this source pool and applies the seven fixed offline tutoring designs. It checks whether CP-MissingBridgeBench can support larger-scale aggregate review on naturally occurring tutoring contexts. It does not evaluate the online AIChat system, does not change student-visible responses, and does not report learning outcomes.

The v2 closeout is reported as an expanded benchmark/evaluation layer and is not merged into the original dialogue-state v3 primary tables. Any future replay or deployment-readiness study should be registered and reported separately, with its own privacy, consent/reporting, selection-bias, and double-review checks.

The observed current-system response is the current system response already shown by online AIChat. It must not be described as a baseline, experimental comparator, control arm, online comparison arm, or Repair output. It is an observed response used to test whether the benchmark rubric can be applied to real dialogue-state cases.

The real AIChat layers do not add a dialogue-state v3 main experiment design, do not recompute dialogue-state tables, do not evaluate learning outcomes, and do not validate deployed-system advantage. Before consent/reporting gates are complete, public reporting is limited to aggregate counts, review integrity, design-level aggregate results, workflow status, schema readiness, and field-sufficiency observations. Case-level labels and examples remain non-reportable.

References

- Arzu Atasoy and Saieed Moslemi Nezhad Arani. ChatGPT: A reliable assistant for the evaluation of students' written texts? *Education and Information Technologies*, 30(14):20385–20415, 2025. doi: 10.1007/s10639-025-13553-1. URL <https://doi.org/10.1007/s10639-025-13553-1>. DOI: 10.1007/s10639-025-13553-1; URL: <https://doi.org/10.1007/s10639-025-13553-1>.
- Konstantina Chrysafiadi, Maria Virvou, George A. Tsihrintzis, and Ioannis Hatzilygeroudis. Evaluating the user's experience, adaptivity and learning outcomes of a fuzzy-based intelligent tutoring system for computer programming for academic students in Greece. *Education and Information Technologies*, 28(6):6453–6483, 2023. doi: 10.1007/s10639-022-11444-3. URL <https://doi.org/10.1007/s10639-022-11444-3>.

- 022-11444-3. DOI: 10.1007/s10639-022-11444-3; URL: <https://doi.org/10.1007/s10639-022-11444-3>.
- Xin Gong, Zhixia Li, and Ailing Qiao. Impact of generative AI dialogic feedback on different stages of programming problem solving. *Education and Information Technologies*, 30(7):9689–9709, 2025. doi: 10.1007/s10639-024-13173-1. URL <https://doi.org/10.1007/s10639-024-13173-1>. DOI: 10.1007/s10639-024-13173-1; URL: <https://doi.org/10.1007/s10639-024-13173-1>.
- Flor de Lis González-Mujico. Measuring student and educator digital competence beyond self-assessment: Developing and validating two rubric-based frameworks. *Education and Information Technologies*, 29(11):13299–13324, 2024. doi: 10.1007/s10639-023-12363-7. URL <https://doi.org/10.1007/s10639-023-12363-7>. DOI: 10.1007/s10639-023-12363-7; URL: <https://doi.org/10.1007/s10639-023-12363-7>.
- Hacer Güner and Erkan Er. AI in the classroom: Exploring students’ interaction with ChatGPT in programming learning. *Education and Information Technologies*, 30(9):12681–12707, 2025. doi: 10.1007/s10639-025-13337-7. URL <https://doi.org/10.1007/s10639-025-13337-7>. DOI: 10.1007/s10639-025-13337-7; URL: <https://doi.org/10.1007/s10639-025-13337-7>.
- Kai Guo and Deliang Wang. To resist it or to embrace it? examining ChatGPT’s potential to support teacher feedback in EFL writing. *Education and Information Technologies*, 29(7):8435–8463, 2024. doi: 10.1007/s10639-023-12146-0. URL <https://doi.org/10.1007/s10639-023-12146-0>. DOI: 10.1007/s10639-023-12146-0; URL: <https://doi.org/10.1007/s10639-023-12146-0>.
- Yuheng Li, Zirui Shan, Mladen Raković, Quanlong Guan, Dragan Gašević, and Guanliang Chen. When AI explains in natural language: Unveiling the impact of generative AI explanations on educators’ grading and feedback practices. *Education and Information Technologies*, 30(17):24931–24964, 2025. doi: 10.1007/s10639-025-13741-z. URL <https://doi.org/10.1007/s10639-025-13741-z>. DOI: 10.1007/s10639-025-13741-z; URL: <https://doi.org/10.1007/s10639-025-13741-z>.
- Shuai Ma, Junling Wang, Yuanhao Zhang, Xiaojuan Ma, and April Yi Wang. DBox: Scaffolding algorithmic programming learning through learner-LLM co-decomposition. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–20, 2025. doi: 10.1145/3706598.3713748. URL <https://doi.org/10.1145/3706598.3713748>. DOI: 10.1145/3706598.3713748; URL: <https://doi.org/10.1145/3706598.3713748>.
- Kaushal Kumar Maurya, Kv Aditya Srivatsa, Kseniia Petukhova, and Ekaterina Kochmar. Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1234–1251, 2025. doi: 10.18653/v1/2025.naacl-long.57. URL <https://aclanthology.org/2025.naacl-long.57/>. DOI: 10.18653/v1/2025.naacl-long.57; URL: <https://aclanthology.org/2025.naacl-long.57/>.
- Huanhuan Wang, Ahmed Tlili, Ronghuai Huang, Zhenyu Cai, Min Li, Zui Cheng, Dong Yang, Mengti Li, Xixian Zhu, and Cheng Fei. Examining the applications of intelligent tutoring systems in real educational contexts: A systematic literature review from the social experiment perspective. *Education and Information Technologies*, 28(7):9113–9148, 2023. doi: 10.1007/s10639-022-11555-x. URL <https://doi.org/10.1007/s10639-022-11555-x>. DOI: 10.1007/s10639-022-11555-x; URL: <https://doi.org/10.1007/s10639-022-11555-x>.

Jin Zhao, Marta Knežević, and Tanja Käser. Evaluating answer leakage robustness of LLM tutors against adversarial student attacks, 2026. URL <https://arxiv.org/abs/2604.18660>. Preprint, arXiv:2604.18660, <https://arxiv.org/abs/2604.18660>.